

UERC20 Factory Audit



Uniswap

June 18, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Conclusion	6
Appendix	7
Issue Classification	7

Summary

Type	DeFi	Total Issues	0 (0 resolved)
Timeline	From 2026-06-01 To 2026-06-05	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	0 (0 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of [PR #64](#) in the [Uniswap/uerc20-factory](#) repository at commit [de5bacd](#).

In scope were the following files:

```
src
├─ factories
│   └─ USUPERC20Factory.sol
└─ libraries
    └─ UERC20MetadataLibrary.sol
```

System Overview

The UERC20 Factory contracts provide deterministic ERC-20 token deployment using CREATE2. Each token address is derived from the token's core identity fields and a caller-provided `graffiti` value, allowing token addresses to be computed before deployment.

The suite includes two token implementations:

- `UERC20`: a standard ERC-20 token with EIP-2612 permit support, ERC-165 interface detection, creator attribution, and optional token metadata.
- `USUPERC20`: an `IERC7802`-compatible token for the Superchain. Supply is initially minted on the home chain, metadata is stored on the home chain, and cross-chain minting and burning are restricted to the Superchain Token Bridge.

The PR under review introduces a generic `extraData` field in the token metadata. This new field holds opaque, application-defined data that is emitted on token creation and stored on-chain. It is not included in the rendered `tokenURI` JSON, and is intended for off-chain verification.

Conclusion

The PR under review replaced the `xProofTweetId` field in the `UERC20Metadata` struct with a generic `extraData` field that is emitted on the `TokenCreated` event. No issues were identified during the audit.

We thank the Uniswap team for their continuous commitment to security across projects.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.